

STUDENT HACKATHON

REQUIREMENTS DOCUMENT

Topic 1

AI-Assisted UI Generation from Wireframe, Code, and Prompt

Mandatory stack: [React.js](#) | [Node.js](#)

Document version 1.0 | August 2026

Audience: Student participants, mentors, and judges

Note: All brand names, IDs, file paths, storage URLs, commit hashes, and production identifiers in this document are fictional sample data. Do not use real client, environment, or credential values.

1. Document Control

Field	Value
Document title	Hackathon Requirements: UI Generation (React.js + Node.js)
Topic code	T1-UI-GEN
Version	1.0
Date	18 August 2026
Status	Ready for student distribution
Classification	Internal hackathon use — sample data only
Tech constraint	Frontend: React.js. Backend: Node.js. Both are mandatory.

1.1 Purpose of this document

This document defines the problem, scope, data contracts, functional requirements, evaluation rubric, and sample artefacts for a student hackathon. Teams will build a system that generates CMS-ready React UI sections from one or more of three inputs: a wireframe image, existing source code, and a natural-language prompt.

1.2 How to use this document

- Organisers: treat Sections 1–7 as briefing material and Sections 16–18 as judging guidance.
- Students: treat Sections 8–15 as the build contract. The appendices are the expected output shape.
- Mentors: use the data model and component contract to review whether generated UI is CMS-editable, not only visually similar.

2. Executive Summary

Modern websites are no longer static pages. Marketing and product teams need sections that can be edited in a CMS after generation: headlines, images, stats, and buttons must have stable element IDs, default fallback content, and a Redux (or equivalent) data binding layer.

The hackathon challenge is to build a generator — not a one-off landing page. Given a wireframe, a code snippet, a prompt, or a combination of these, the system must emit:

1. A React section component that follows the sample architecture in Appendix C.
2. A section metadata record (Appendix A).
3. A list of CMS element records (Appendix B).
4. A live preview of the generated section.

Success is measured by layout fidelity, CMS contract compliance, code quality, and the ability to regenerate or vary a section without hand-rewriting IDs and bindings.

3. Problem Statement

Converting a design into a production-ready, content-managed React section is slow and inconsistent. Typical issues:

- Wireframes describe layout, but developers invent different class names, spacing, and component structure.
- Prompts describe intent, but generated JSX is often presentational only — hard-coded text, no IDs, no CMS fetch.
- Existing section code is difficult to reuse because IDs, Redux selectors, and default content are tightly coupled.
- Repeatable blocks (stat cards, feature lists) need a loop schema, not three unrelated hard-coded blocks.

Students must close this gap: an automated pipeline that understands visual structure, textual intent, and code patterns, then outputs a section that a CMS editor could later change without touching React source.

4. Hackathon Objectives

4.1 Primary objectives (must achieve)

- Generate a visually faithful React section from at least two of the three input types (wireframe, code, prompt).
- Bind every editable field to a unique fieldId and render CMS content with a hard-coded default fallback.
- Persist section metadata and element documents in a Node.js API with a document store (MongoDB, or a JSON file store if MongoDB is unavailable).
- Preview the generated section in the browser using Redux (or equivalent) as the runtime content source.

4.2 Secondary objectives (strongly recommended)

- Accept combined inputs (example: wireframe + prompt that says “make CTA green and use 3 stat cards”).
- Support a second variation of the same section (variations field in section metadata).
- Re-generate from existing code while preserving element IDs when the layout change is small.

4.3 Stretch objectives (bonus)

- Vision model that labels regions of the wireframe (hero image, badge, H1, stats, CTA).
- CSS overlay per fieldId (allSectionsCss) applied after mount.
- Export a zip of the generated component plus seed JSON for section and elements.
- Simple content editor in the preview that writes back to the element store.

5. Scope

5.1 In scope

- A student-built generator UI (React) and generator API (Node.js).
- One reference page: Home. One reference section: a split hero (image left, content right).
- Element types: Image, Text, Textfield, Button, Cards (with loop items), carousel, wizards, etc.
- Responsive layout using Tailwind CSS utility classes.
- PrimeReact Button (or equivalent accessible button) for the primary CTA.
- Placeholder images and fictional brand copy only.

5.2 Out of scope

- Real client brands, real production databases, real storage credentials, and real commit / deploy pipelines.
- Full multi-tenant CMS, authentication SSO, payments, or analytics.
- Native mobile apps (platform for this topic is Website).
- Pixel-perfect Figma plugin work unless a team chooses it as a stretch extra.
- Training a foundation model from scratch.

6. Target Users and Personas

Persona	Goal during the hackathon	Success looks like
Student builder	Ship a working generator + preview	Judges can upload a wireframe and see a CMS-bound section
Mock content editor	Change headline / image without editing JSX	Updating an element record updates the preview
Mentor / judge	Score contract compliance, not only beauty	IDs, fallbacks, cards loop, and APIs are demonstrable

7. Mandatory Technology Stack

The following stack is required unless a judge grants a written exception before the event.

Layer	Required	Allowed alternatives / notes
Frontend framework	React.js (functional components)	Vite recommended. Create React App is acceptable.
UI styling	Tailwind CSS for layout	May combine with a small amount of CSS modules.
Component library	PrimeReact for Button (and optional form widgets)	If PrimeReact fails to install, use a semantic <code><button></code> with equivalent classes.
Client state	Redux Toolkit for CMS section/element maps	Zustand allowed only if the same selector shape is documented.
Routing	react-router-dom	At least <code>/preview/:pageName</code> and <code>/generate</code> .
Backend runtime	Node.js	Express, Fastify, or NestJS are all acceptable.
Persistence	MongoDB or file-based JSON store	Must expose REST (or GraphQL) for sections and elements.
File upload	Multipart upload for wireframe images	Store files locally under <code>/uploads</code> during the event.
AI / LLM	Any hosted or local model	Must not leak API keys in the repo. Use <code>.env</code> (not committed).
Language	JavaScript or TypeScript	TypeScript is preferred but not mandatory.

7.1 Environment variables (sample names only)

Teams must document these names. Do not commit secret values.

```
VITE_STORAGE_URL=http://localhost:4000/storage/  
VITE_API_URL=http://localhost:4000/api  
MONGODB_URI=mongodb://localhost:27017/hackathon_ui_gen  
LLM_API_KEY=replace-me  
LLM_BASE_URL=https://example-llm-provider.local/v1
```

8. Challenge Definition

Teams build a UI Generation Studio. A user supplies one or more inputs. The studio returns a React section that matches the visual structure and is wired for CMS editing.

8.1 Input mode A — Wireframe

The user uploads a PNG/JPG/WebP of a section wireframe (low-fidelity boxes or a screenshot-style mock). The system infers regions: media, eyebrow/badge, primary heading, subheading, body copy, repeating stats/cards, and primary action.

- Accept at least PNG and JPG.
- Show the uploaded wireframe beside the generated preview for visual comparison.
- If a region is uncertain, still emit a plausible element with a default and mark confidence in the API response (optional but scored).

8.2 Input mode B — Existing code

The user pastes a React section (or a simplified JSX tree). The system must parse structure, extract text/image/button nodes, assign or reuse fieldIds, and emit a CMS-compliant component plus element JSON. This mode tests understanding of the contract, not only image-to-layout.

- Detect hard-coded strings and lift them into element records with defaults.
- Detect repeating children and convert them into a Cards loop.
- Preserve Tailwind layout classes when they already express the wireframe.

8.3 Input mode C — Prompt

The user types intent, for example: “Create a fitness hero. Left image of an athlete. Right side: red uppercase badge, bold headline, supporting paragraph, three stats, full-width red CTA.” The system must produce the same contract as modes A and B.

- Prompt-only generation is valid, but layout should still be responsive (stacked on mobile, split on desktop).
- The prompt may override colours, copy, number of cards, and CTA label.

8.4 Combined inputs (recommended)

Priority when inputs conflict:

1. Prompt instructions win for copy, colour, and CTA behaviour.

2. Wireframe wins for spatial layout (columns, order, alignment).
3. Existing code wins for technical patterns (Redux selectors, helper names, class conventions) unless the prompt asks to change them.

9. Reference User Journey

1. Open the Generator Studio at /generate.
2. Choose input: Wireframe, Code, Prompt, or Combine.
3. Optionally set pageName (default Home) and sectionName (default Custom).
4. Submit. The Node API runs analysis + generation.
5. API writes a section document and N element documents, then returns generated JSX (or a component file path) plus IDs.
6. Frontend loads /preview/Home, dispatches fetchElementsByIds, and renders the section.
7. User may edit an element in a side panel; preview updates from store data, not from re-generating JSX.

10. System Architecture

Logical components (teams may merge files but should keep these responsibilities visible):

Component	Runtime	Responsibility
Generator Studio UI	React	Capture wireframe, code, prompt; show job status and preview
Preview shell	React + Redux	Provide cms.allSections and cms.allSectionsCss; mount generated section
Generate API	Node.js	Orchestrate parse → model → validate → persist → return artefact
Vision / layout parser	Node.js + optional LLM	Turn image or JSX into a layout IR (intermediate representation)
Code synthesiser	Node.js + optional LLM	Emit React component that obeys the contract in Section 12
Section store	MongoDB / JSON	CRUD for section metadata
Element store	MongoDB / JSON	CRUD for CMS fields, including Cards.loop
Static storage	Local disk	Wireframes and placeholder images served under /storage

10.1 Suggested intermediate representation (IR)

Before emitting JSX, convert inputs into a layout IR. This makes validation and tests possible even if the LLM output varies.

```
{
  "sectionType": "split-hero",
  "platform": "Website",
  "layout": {
    "direction": "row",
    "breakpoint": "md",

```

```

    "left": { "role": "media", "element": "heroImage" },
    "right": ["brandBadge", "headlineMain", "headlineSub", "description", "statBadges", "ctaButton"]
  },
  "theme": { "accent": "red-500", "surface": "white", "text": "gray-800" },
  "elements": [ /* see Appendix B */ ]
}

```

11. Domain Model and Data Contracts

The data shapes below are the source of truth for persistence and for the generated React bindings. Field names should be kept so judges can inspect documents quickly. Values are fictional.

11.1 Identifiers

- `sectionId`: 10-digit string unique within the project (example 1000000001).
- `fieldId`: 10-digit string unique within the project (example 2000000001).
- Nested card fields use `fieldId1`, `fieldId2`, ... on each loop item.
- `pageName`: human page key used in Redux, example Home.
- Do not expose real ObjectIds, cloud URLs, or git commit hashes in demos or repos.

11.2 Section document

One document per generated section instance. Key fields:

Field	Type	Meaning
<code>sectionName</code>	string	Logical type, e.g. Custom, Hero, Features
<code>sectionId</code>	string	Stable ID referenced by elements
<code>variations</code>	string/number	How many layout variants exist
<code>path</code>	string	Virtual component path, not a real client folder
<code>sectionStatus</code>	string	Pending Approved Rejected
<code>wireframes</code>	string	Relative path to the source wireframe, if any
<code>platform</code>	string	Website for this topic
<code>pageName</code>	string	Page the section belongs to
<code>isGenerated</code>	boolean	true when produced by the generator
<code>cardGridColumns</code>	number	Default 3 for stats/cards
<code>cardLayoutMode</code>	string	grid list
<code>sectionTextMode</code>	string	auto light dark
<code>sectionColor / paddings</code>	string	Optional theme tokens applied on the wrapper

11.3 Element document

Field	Type	Meaning
<code>sectionId</code>	string	Parent section
<code>elementName</code>	string	Stable semantic name: heroImage, headlineMain, ...

fieldId	string	DOM id and Redux key
content	string	Default / current CMS value
contentType	enum	Image Text Textfield Button Cards
css	string null	Inline cssText overlay for this fieldId
loop	array	Required when contentType is Cards
projectName	string	Use sample-brand only
pageName	string	Must match section.pageName

11.4 Cards loop item

Each loop item is a small record with paired fields. The sample hero uses two text fields per card (value + label). Teams may add field3 if the wireframe shows icons, but the reference implementation uses two fields.

```
{
  "field1": "1000+",
  "fieldType1": "Text",
  "fieldId1": "3000000001",
  "field2": "Community<br />Members",
  "fieldType2": "Text",
  "fieldId2": "3000000002"
}
```

11.5 Runtime Redux shape

```
state.cms.allSections[pageName][fieldId] = contentStringOrLoopArray
state.cms.allSectionsCss[pageName][fieldId] = cssTextString
state.cms.sectionNames = { [sectionId]: sectionName }
```

The generated component must read from this shape. Direct fetch-inside-JSX without a store is incomplete for this challenge.

12. Generated Component Contract

Every generated section MUST follow these rules. Judges will open the file and the DOM.

12.1 Mandatory rules

- R1. Declare a const ids object mapping semantic names to fieldId strings.
- R2. Accept pageName as a prop with a sensible default (Home).
- R3. On mount, dispatch fetchElementsByIds with every fieldId used in the tree, including nested card field IDs.
- R4. Read live values from state.cms.allSections[pageName].
- R5. Every editable node has id={ids.something} or id={item.fieldIdN}.
- R6. Text nodes use dangerouslySetInnerHTML with a default fallback: data?.[id] || "DEFAULT".
- R7. Images use a getImage helper, VITE_STORAGE_URL prefix, blob: passthrough, and onError placeholder.
- R8. Buttons use PrimeReact Button (or documented fallback) with label from CMS and an onClick stub.

- R9. Repeating items render from a loop array with a `DEFAULT_*` constant when CMS data is missing or the wrong length.
- R10. Apply `allSectionsCss` to matching DOM ids after `cssData` changes.
- R11. Use Tailwind for layout. Desktop: two columns. Mobile: stacked. Include a max-width container.
- R12. Add class `dynamicStyle` on text/button nodes and `dynamicStyle2` on images so a future editor can hook them.
- R13. Do not embed real secrets, real bucket URLs, or real customer identifiers.
- R14. Export default the section component.

12.2 Accessibility and quality rules

- Images have meaningful alt text derived from the prompt or a generic description.
- CTA has aria-label.
- Colour contrast: body text gray-500+ on white is acceptable; do not put gray-400 on white for long copy.
- Do not leave empty id attributes on nested card fields (the sample in Appendix C includes ids on both stat value and label).

12.3 What generated code must NOT do

- Hard-code production storage hosts.
- Import internal proprietary helpers that students cannot implement (replace with local utils).
- Skip Redux and only render defaults.
- Use `Math.random()` for field IDs on every render (IDs must be stable and persisted).

13. Functional Requirements

Requirements are numbered for traceability. Priority: Must / Should / Could.

13.1 Generator Studio

ID	Pri	Requirement
FR-G01	Must	Provide a form to upload a wireframe image.
FR-G02	Must	Provide a textarea to paste existing React/JSX code.
FR-G03	Must	Provide a textarea for a natural-language prompt.
FR-G04	Must	Allow selecting one mode or combining two+ inputs in a single job.
FR-G05	Must	Show generation progress and surface API errors in plain language.
FR-G06	Must	After success, show generated JSX (read-only) and a link to preview.
FR-G07	Should	Let the user set <code>pageName</code> , <code>sectionName</code> , and accent colour.
FR-G08	Should	Keep the last 5 jobs in a history list (in-memory is enough).
FR-G09	Could	Download a zip: <code>Component.jsx</code> + <code>section.json</code> + <code>elements.json</code> .

13.2 Generation engine

ID	Pri	Requirement
FR-E01	Must	Produce unique 10-digit fieldIds for new elements and persist them.
FR-E02	Must	Map detected regions to the reference element set for the hero: heroImage, brandBadge, headlineMain, headlineSub, description, statBadges, ctaButton.
FR-E03	Must	Create Cards.loop with exactly the count requested (default 3) and unique nested field IDs.
FR-E04	Must	Emit JSX that compiles in the preview app without manual edits.
FR-E05	Must	Include default fallback copy for every text/button/image.
FR-E06	Should	If code input already contains an ids map, reuse those IDs (stable regeneration).
FR-E07	Should	Validate IR before synthesis: missing CTA or missing heading is a warning, not a silent drop.
FR-E08	Could	Return a confidence score per element when the source is a wireframe.

13.3 Preview and CMS runtime

ID	Pri	Requirement
FR-P01	Must	Render the generated section inside a preview route.
FR-P02	Must	Hydrate Redux from GET /elements?pageName=Home (or equivalent).
FR-P03	Must	Changing an element via API or a small editor updates the preview without regenerating JSX.
FR-P04	Must	Broken image paths fall back to the placeholder.
FR-P05	Should	Responsive preview: toggle mobile (~375px) and desktop (~1280px) widths.
FR-P06	Should	Apply per-element css strings from the store onto DOM ids.
FR-P07	Could	Side-by-side wireframe vs live preview slider.

13.4 Persistence APIs

ID	Pri	Requirement
FR-A01	Must	POST /api/generate accepts wireframe file and/or code and/or prompt.
FR-A02	Must	GET /api/sections and GET /api/sections/:sectionId.
FR-A03	Must	GET /api/elements?sectionId= or ?pageName=
FR-A04	Must	PATCH /api/elements/:fieldId to update content and css.
FR-A05	Should	POST /api/sections/:sectionId/regenerate for variation 2.
FR-A06	Should	Health endpoint GET /api/health.

14. API Specification (minimum)

14.1 POST /api/generate

Content-Type: multipart/form-data

Part	Required	Description
mode	Yes	wireframe code prompt combined
prompt	If mode needs it	Natural language instructions
code	If mode needs it	Existing React/JSX as text
wireframe	If mode needs it	Image file
pageName	No	Default Home
sectionName	No	Default Custom

Sample success response:

```
{
  "ok": true,
  "sectionId": "1000000001",
  "pageName": "Home",
  "componentFile": "src/sections/generated/HeroSection.jsx",
  "elementIds": ["2000000001", "2000000002", "2000000003"],
  "warnings": ["Subheading inferred with low confidence"],
  "ir": { "sectionType": "split-hero" }
}
```

14.2 GET /api/elements

Returns the array used to fill Redux. Nested Cards must include loop. Text content may contain simple HTML (span, br, strong) because the React contract uses dangerouslySetInnerHTML.

14.3 Error model

```
{
  "ok": false,
  "error": {
    "code": "INVALID_INPUT",
    "message": "At least one of wireframe, code, or prompt is required."
  }
}
```

Use HTTP 400 for validation, 413 for oversized images (recommend max 8 MB), 422 for model/parse failure, 500 for unexpected errors.

15. Non-Functional Requirements

ID	Area	Requirement
NFR-01	Performance	Preview first paint of a generated section under 2s on local after data is cached.
NFR-02	Performance	Generate API should respond within 60s; show a spinner if the LLM is

		slower.
NFR-03	Reliability	If the LLM fails, return 422 with a message; do not write partial IDs without a section row.
NFR-04	Security	No secrets in git. Sanitize HTML from CMS to a small tag allow-list if time allows.
NFR-05	Security	Do not execute user-pasted code on the server (parse as text / AST only).
NFR-06	Privacy	Use only fictional brand copy. Strip any real names found in pasted code before saving.
NFR-07	Usability	Empty states and inline validation on the generate form.
NFR-08	Maintainability	README with setup, .env.example, and a 3-step demo script for judges.
NFR-09	Compatibility	Chrome or Edge latest. Node 18+ recommended (document if constrained).
NFR-10	Accessibility	Keyboard reachable generate form; preview CTA focus ring visible.

Note: `dangerouslySetInnerHTML` is part of the CMS contract. Teams should still avoid introducing script tags. A simple allow-list (*b*, *i*, *br*, *span*, *strong*, *em*) is a judged plus.

16. Sample Scenario (for the event brief)

A fitness brand needs a Home page hero. Design has provided a split layout: athlete image on the left; on the right a red eyebrow, large headline, supporting line, short paragraph, three statistics, and a primary button. Content should be CMS-editable after launch.

16.1 Sample prompt (give this to students)

Create a full-viewport website hero for a fitness brand named Pulse Fit.
 Layout: two columns on desktop, stacked on mobile. Left: product / athlete image.
 Right: uppercase red badge, extra-bold headline, muted subheading, body paragraph,
 three stat cards in a row (value + label), and a red filled CTA button.
 Use white background, red accent bars on far left and far right for large screens.
 All text, image, stats, and button must be CMS-bound with stable IDs and defaults.

16.2 Sample default copy

Element	Default content
brandBadge	PULSE FIT
headlineMain	CHALLENGE YOUR LIMITS
headlineSub	Be a part of the tribe that's limitless.
description	Join trainer-led workout sessions designed to kickstart your fitness journey, at your convenience.
stat 1	1000+ / Community Members
stat 2	40+ / Fitness Programmes

stat 3	150+ / Fitness Channels
ctaButton	FIND A WORKOUT
heroImage alt	Athlete performing a dumbbell exercise

17. Deliverables

Each team submits:

1. Source repository (Git) with React app and Node API.
2. README: setup, how to run, how to demo each input mode, known limitations.
3. .env.example with dummy values only.
4. At least one generated section checked in under src/sections/generated/ as a reference output.
5. Seed JSON for one section and its elements (sanitized sample data).
6. Short demo script (5–8 minutes) covering wireframe, prompt, and code inputs if implemented.
7. Optional: screenshots or a screen recording of preview vs wireframe.

Do not submit: production dumps, real customer JSON, API keys, or internal commit hashes.

18. Suggested Timeline (8-hour and 24-hour variants)

18.1 Eight-hour sprint

Hours	Focus
0.0 – 0.5	Read this document. Split roles: API, generator UI, preview/contract.
0.5 – 2.0	Scaffold React + Node. Redux cms slice. Dummy HeroSection from Appendix C.
2.0 – 4.0	POST /api/generate for prompt mode. Persist section + elements. Wire preview.
4.0 – 6.0	Wireframe upload + IR. Improve layout fidelity. Cards loop.
6.0 – 7.0	Code-input path OR content editor + PATCH element.
7.0 – 8.0	README, seed data, dry-run demo, remove secrets.

18.2 Twenty-four-hour sprint (extra)

- Add variation 2, css overlays, zip export, HTML sanitiser, mobile/desktop preview frames, and confidence scores.

19. Evaluation Rubric

Total 100 points. Partial credit is allowed. A visually pretty page that is not CMS-bound cannot score above 40.

Criterion	Points	What judges look for
CMS contract compliance	25	ids map, fetch on mount, fallbacks, fieldIds in DOM, Cards loop, css overlay hook
Input coverage	20	Working prompt mode (required). Wireframe and/or code

		mode. Combined inputs bonus.
Layout fidelity	15	Split hero, stats row, CTA, responsive stack, accent treatment
Backend quality	15	Generate + CRUD APIs, persistence, validation, no secrets
Code quality	10	Readable React/Node, README, .env.example, no dead proprietary imports
UX of the studio	10	Clear form, errors, preview, optional editor
Innovation / stretch	5	IR, sanitiser, variations, zip export, vision labels

19.1 Disqualification / major penalties

- Submitting real client data, secrets, or production URLs: 0 for security and possible disqualification.
- Frontend not in React.js or backend not in Node.js: topic requirement failed (cap at 20 unless exception granted).
- Preview only shows static HTML with no store/API: cap at 40.
- Plagiarism of another team's generator: disqualification.

20. Rules and Constraints

- Teams of 2–4 unless the organiser states otherwise.
- Open-source libraries are allowed. Document them in README.
- LLM APIs are allowed. Teams must run with a missing key in a degraded way (clear error), not a crash.
- Internet may be limited; a prompt-template fallback that still emits the contract is valued.
- All demo content must be fictional (Pulse Fit / sample-brand).
- Mentors may clarify requirements but will not write the generator for a team.

21. Setup Expectations for Organisers

- Provide this Word document plus a sample wireframe PNG (boxes labelled Hero image / Badge / Headline / Stats / CTA).
- Provide Appendix C as a starting preview component if you want a baseline compile.
- Provide a blank React+Node starter only if time is short; otherwise scaffolding is part of the challenge.
- Judges should have Node and a browser; MongoDB Atlas or local Mongo is optional if JSON store is accepted.

22. Risks and Mitigations

Risk	Mitigation
LLM output is invalid JSX	Validate with a parser; retry once; fall back to a template filled from IR.
IDs collide across jobs	Allocate IDs in the API (central counter or nanoid numeric), never in the LLM.

Students paste proprietary code	Brief them to use Appendix C only; add a pre-submit grep for forbidden hostnames.
Images 404	Ship a default placeholder under /storage/default/images/.
PrimeReact install issues	Allow native button with the same id/class/label contract.

23. Glossary

Term	Meaning in this hackathon
Section	A full-width page block (e.g. hero) with metadata and many elements
Element	One CMS-editable field (text, image, button) with a fieldId
Cards / loop	A repeating group stored as an array of field pairs
IR	Intermediate representation of layout before JSX is written
dynamicStyle	Marker class for text/button nodes hooked by an editor
dynamicStyle2	Marker class for image nodes
Variation	An alternate layout of the same sectionId family
Preview	Runtime that hydrates CMS data into the generated React tree

24. Acceptance Checklist (students self-score before demo)

- ☐ Prompt mode generates a split hero that compiles.
- ☐ At least one of: wireframe mode or code mode works end-to-end.
- ☐ Section JSON and element JSON are stored and retrievable.
- ☐ DOM shows the expected ids (inspect heroImage, headlineMain, ctaButton).
- ☐ Changing headline via PATCH updates preview without code edit.
- ☐ Three stat cards render from loop with nested field IDs.
- ☐ Image fallback works when src is empty.
- ☐ README lists exact run commands.
- ☐ No API keys or real client names in the repo.
- ☐ Mobile stack and desktop split both demonstrated.

Appendix A — Sample Section Document (sanitized)

Use this as seed data. All identifiers and paths are fictional. Organisers may load this into MongoDB or a JSON file.

```
{
  "_id": { "$oid": "aaaaaaaaaaaaaaaaaaaaaa" },
  "sectionName": "Custom",
  "sectionId": "1000000001",
  "variations": "2",
  "path": "/client/sample-brand/Custom/CustomHero",
  "sectionStatus": "Pending",
  "wireframes": "sample/wireframes/hero-section.png",
  "platform": "Website",
  "createdAt": { "$date": "2026-07-03T12:27:09.463Z" },
  "updatedAt": { "$date": "2026-07-03T12:27:09.463Z" },
  "pageId": { "$oid": "bbbbbbbbbbbbbbbbbbbbbb" },
  "pageName": "Home",
  "isGenerated": true,
  "layoutPlacementId": "ly-000000000000-sample",
  "isBuild": true,
  "cardClassName": "",
  "cardCssText": "",
  "cardGridColumns": 3,
  "cardLayoutMode": "grid",
  "cardListBodyGapPx": 15,
  "sectionClassName": "",
  "sectionColor": "",
  "sectionPaddingBottom": "",
  "sectionPaddingLeft": "",
  "sectionPaddingRight": "",
  "sectionPaddingTop": "",
  "sectionTextMode": "auto",
  "tableClassName": "",
  "tableCssText": ""
}
```

Appendix B — Sample Element Documents (sanitized)

Seven elements for the reference hero. Cards includes a loop of three items. Optional fields such as timestamps may be added by the API.

```
[
  {
    "_id": { "$oid": "cccccccccccccccccccccc" },
    "sectionId": "1000000001",
    "elementName": "heroImage",
    "fieldId": "2000000001",
    "content": "default/images/hero-placeholder.jpg",
    "contentType": "Image",
    "section": "Custom",
    "projectName": "sample-brand",
    "pageId": { "$oid": "bbbbbbbbbbbbbbbbbbbbbb" },
    "pageName": "Home",
    "isCustom": true,
    "css": null
  },
  {
    "elementName": "brandBadge",
    "fieldId": "2000000002",
    "content": "PULSE FIT",
    "contentType": "Text"
  },
  {
    "elementName": "headlineMain",
    "fieldId": "2000000003",
    "content": "CHALLENGE YOUR LIMITS",
  }
]
```



```

    "contentType": "Text",
    "css": "font-weight: bold; text-align: left;"
  },
  {
    "elementName": "headlineSub",
    "fieldId": "2000000004",
    "content": "Be a part of the tribe that's limitless.",
    "contentType": "Text"
  },
  {
    "elementName": "description",
    "fieldId": "2000000005",
    "content": "Join trainer-led workout sessions designed to kickstart your fitness journey, at your convenience.",
    "contentType": "Textfield"
  },
  {
    "elementName": "statBadges",
    "fieldId": "2000000006",
    "content": "",
    "contentType": "Cards",
    "loop": [
      {
        "field1": "100+",
        "fieldType1": "Text",
        "fieldId1": "3000000001",
        "field2": "Community<br />Members",
        "fieldType2": "Text",
        "fieldId2": "3000000002"
      },
      {
        "field1": "40+",
        "fieldType1": "Text",
        "fieldId1": "3000000003",
        "field2": "Fitness<br />Programmes",
        "fieldType2": "Text",
        "fieldId2": "3000000004"
      },
      {
        "field1": "150+",
        "fieldType1": "Text",
        "fieldId1": "3000000005",
        "field2": "Fitness<br />Channels",
        "fieldType2": "Text",
        "fieldId2": "3000000006"
      }
    ]
  },
  {
    "elementName": "ctaButton",
    "fieldId": "2000000007",
    "content": "FIND A WORKOUT",
    "contentType": "Button"
  }
]

```

Appendix C — Sample Generated React Section (sanitized)

This is the expected shape of generator output. Proprietary imports, real storage hosts, real brand names, and production IDs have been removed. Students may paste this into the preview app as a baseline, then replace it with generator output.

```

import React, { useEffect, useRef } from "react";
import { useSelector, useDispatch } from "react-redux";
import { Button } from "primereact/button";
import { fetchElementsByIds } from "../../redux";
import { getSectionTextContrastClass } from "../../utils/sectionContrast";

```

```

// CMS element ids created by the generator
const ids = {
  heroImage: "2000000001",
  brandBadge: "2000000002",
  headlineMain: "2000000003",
  headlineSub: "2000000004",
  description: "2000000005",
  statBadges: "2000000006",
  ctaButton: "2000000007",
};

const DEFAULT_STAT_CARDS = [
  {
    field1: "100+",
    fieldType1: "Text",
    fieldId1: "3000000001",
    field2: "Community<br />Members",
    fieldType2: "Text",
    fieldId2: "3000000002",
  },
  {
    field1: "40+",
    fieldType1: "Text",
    fieldId1: "3000000003",
    field2: "Fitness<br />Programmes",
    fieldType2: "Text",
    fieldId2: "3000000004",
  },
  {
    field1: "150+",
    fieldType1: "Text",
    fieldId1: "3000000005",
    field2: "Fitness<br />Channels",
    fieldType2: "Text",
    fieldId2: "3000000006",
  },
];

function getImage(image) {
  if (!image) {
    return `${import.meta.env.VITE_STORAGE_URL}default/images/hero-placeholder.jpg`;
  }
  if (image.includes("blob:")) {
    return image;
  }
  return `${import.meta.env.VITE_STORAGE_URL}${image}`;
}

function errorImage(event) {
  event.target.src = `${import.meta.env.VITE_STORAGE_URL}default/images/hero-placeholder.jpg`;
}

const HeroSection = ({ pageName = "Home" }) => {
  const dispatch = useDispatch();
  const data = useSelector((state) => state.cms?.allSections?.[pageName]);
  const cssData = useSelector((state) => state.cms?.allSectionsCss?.[pageName]);
  const rootRef = useRef(null);
  const textContrastClass = getSectionTextContrastClass(undefined, undefined);

  const statBadgesArr =
    Array.isArray(data?.[ids.statBadges]) && data[ids.statBadges].length === 3
      ? data[ids.statBadges]
      : DEFAULT_STAT_CARDS;

  useEffect(() => {
    dispatch(
      fetchElementsByIds({
        elementIds: [
          ids.heroImage,
          ids.brandBadge,
          ids.headlineMain,
          ids.headlineSub,
          ids.description,

```

```

        ids.statBadges,
        "3000000001",
        "3000000002",
        "3000000003",
        "3000000004",
        "3000000005",
        "3000000006",
        ids.ctaButton,
    ],
    pageName,
  ))
);
}, [dispatch, pageName]);

useEffect(() => {
  Object.values(ids).forEach((id) => {
    if (id && cssData?.[id]) {
      const el = document.getElementById(id);
      if (el) el.style.cssText = cssData[id];
    }
  });
}, [cssData]);

const handleCtaClick = () => {
  alert("Find a workout clicked!");
};

return (
  <div
    ref={rootRef}
    className={`relative min-h-screen w-full flex items-center justify-center overflow-hidden bg-background-
color ${textContrastClass}`}
  >
    <div className="hidden md:block absolute left-0 top-0 h-full w-8 bg-red-500 z-0" />
    <div className="hidden md:block absolute right-0 top-0 h-full w-8 bg-red-500 z-0" />

    <main className="relative z-10 w-full max-w-[1920px] mx-auto px-0 md:px-12 flex items-center justify-center
min-h-screen">
      <section className="w-full flex flex-col md:flex-row bg-white overflow-hidden">
        <div className="md:w-1/2 w-full flex items-center justify-center min-h-[400px] md:min-h-[680px] py-8
md:py-16 bg-white">
          <img
            id={ids.heroImage}
            className="dynamicStyle2 max-w-full max-h-[600px] h-auto object-contain mx-auto"
            src={
              data?.[ids.heroImage]
                ? getImage(data[ids.heroImage])
                : `${import.meta.env.VITE_STORAGE_URL}default/images/hero-placeholder.jpg`
            }
            alt="Athlete performing a dumbbell exercise"
            onError={errorImage}
          />
        </div>

        <div className="md:w-1/2 w-full flex flex-col justify-center px-6 md:px-16 py-12 md:py-16 space-y-8 bg-
white">
          <span
            id={ids.brandBadge}
            className="dynamicStyle uppercase text-red-500 font-bold text-lg"
            dangerouslySetInnerHTML={{
              __html: data?.[ids.brandBadge] || "PULSE FIT",
            }}
          />
          <h1
            id={ids.headlineMain}
            className="dynamicStyle text-4xl md:text-5xl font-extrabold tracking-tight text-gray-800 leading-
tight"
            dangerouslySetInnerHTML={{
              __html: data?.[ids.headlineMain] || "CHALLENGE YOUR LIMITS",
            }}
          />
          <h2
            id={ids.headlineSub}

```

```

        className="dynamicStyle text-lg md:text-xl font-medium text-gray-500 tracking-wide"
        dangerouslySetInnerHTML={{
          __html: data?.[ids.headlineSub] || "Be a part of the tribe that's limitless.",
        }}
      />
    <p
      id={ids.description}
      className="dynamicStyle text-gray-500 text-base md:text-lg max-w-xl leading-relaxed"
      dangerouslySetInnerHTML={{
        __html:
          data?.[ids.description] ||
          "Join trainer-led workout sessions designed to kickstart your fitness journey.",
      }}
    />

    <div
      id={ids.statBadges}
      className="dynamicStyle flex flex-col md:flex-row md:justify-between md:gap-x-8 gap-y-8 pt-2"
    >
      {statBadgesArr.map((item) => (
        <div key={item.fieldId1} className="flex-1 flex flex-col items-center justify-center">
          <div
            id={item.fieldId1}
            className="text-3xl md:text-4xl font-extrabold text-gray-800"
            dangerouslySetInnerHTML={{
              __html: data?.[item.fieldId1] || item.field1,
            }}
          />
          <div
            id={item.fieldId2}
            className="mt-1 text-gray-500 text-base"
            dangerouslySetInnerHTML={{
              __html: data?.[item.fieldId2] || item.field2,
            }}
          />
        </div>
      ))}
    </div>

    <Button
      id={ids.ctaButton}
      className="dynamicStyle w-full md:w-auto bg-red-500 hover:bg-red-600 text-white font-semibold
rounded"
      severity="danger"
      aria-label="Find a workout"
      label={data?.[ids.ctaButton] || "FIND A WORKOUT"}
      onClick={handleCtaClick}
    />
  </div>
</section>
</main>
</div>
);
};

export default HeroSection;

```

Appendix D — Mapping: Wireframe regions to elements

Wireframe region (typical)	elementName	contentType	DOM notes
Left large media box	heroImage	Image	id + dynamicStyle2 + getImage + onError
Small uppercase label	brandBadge	Text	span, HTML allowed
Largest heading	headlineMain	Text	h1

Secondary heading / tagline	headlineSub	Text	h2
Paragraph under tagline	description	Textfield	p
Row of 3 metric blocks	statBadges	Cards	container id; children use fieldId1/2
Primary action	ctaButton	Button	PrimeReact Button label bound to CMS

Appendix E — Sample judge demo script (8 minutes)

1. Show README and .env.example (30s). Start API + React.
2. Prompt mode: paste the sample prompt. Show IR or logs. Open preview (2 min).
3. Inspect DOM for ids. PATCH headlineMain to “TRAIN WITHOUT LIMITS” and refresh preview (1.5 min).
4. Wireframe or code mode: generate again. Point out reused or new IDs (2 min).
5. Responsive toggle and one known limitation (1 min).
6. Questions (remaining time).

Appendix F — Data sanitisation policy used in this brief

The following were removed or replaced relative to any internal examples used to design the topic:

- Real project / brand names → sample-brand / Pulse Fit.
- Real MongoDB ObjectIds → placeholder 24-character ids.
- Real 10-digit production field IDs → 1000... / 2000... / 3000... ranges.
- Real storage and CDN URLs → VITE_STORAGE_URL only.
- Git commit hashes, deploy flags, and internal request filenames → omitted or replaced with sample/wireframes/hero-section.png.
- Internal package paths and proprietary helper names → short local equivalents (fetchElementsByIds).
- Employee or machine file paths → not included.

Students and organisers must apply the same policy to anything they add during the event.

End of document — Topic T1-UI-GEN — Version 1.0

For student hackathon use. Sample data only.